

제어 가능한 커스텀 물리 엔진 설계

(부제: 접촉 해결부터 자세 제어, 서버 권위 동기화까지)

My YachtDice 프로젝트 적용 사례

김민용

문서 안내

본 문서는 프로젝트에서 직접 설계 및 구현 내용을 중심으로 작성한 상세 포트폴리오 문서입니다. 전체 포트폴리오에서 요약하여 소개한 내용을 기반으로, 주요 시스템의 설계 의도와 구현 내용, 기술적 판단을 보다 상세하게 설명합니다.

실제 게임 플레이와 주요 시스템의 동작은 아래 영상에서 확인할 수 있습니다.

<https://youtu.be/Szg7PwBPmzQ?si=hCo6EtbVJrK3N3TI> (멀티플레이 환경 플레이)

<https://youtu.be/Bx78MtuWQsE> (YachtDice 345점 플레이 영상)

목차

1. 프로젝트 개요	3
1.1. 개발 배경.....	3
1.2. 주요 개발 항목 및 핵심 과제.....	4
1.3. 프로젝트 정보.....	4
2. 물리 파이프라인 설명 (배경지식 설명).....	5
2.1. 물리 엔진 전체 처리 흐름.....	5
3. 접촉 해결을 안정적으로 처리하는 Contact Solver 만들기	6
3.1. [문제] 물체끼리 접촉할 때 접촉면을 파고드는 문제가 있다.....	6
3.2. [난이도] 화면에 보이는 것은 결과뿐, 어느 부분이 원인인지 특정하기 어렵다.....	7
3.3. [해결 1] 접촉 지점에 대한 반복 계산으로 해결한다. (Iterative Solve).....	9
3.4. [해결 2] 충돌 시점의 속도를 bias로 두어 반발을 유지한다.....	10
3.5. [결과] 파고드는 것이 누적되지 않고 반발이 유지된다.....	11
3.6. [트레이드오프] 안정적인 접촉 처리를 얻는 대신 연산량과 정확도를 내주었다.....	12
4. 물리 운동을 유지하며 목표 자세(Orientation)로 유도하기.....	13
4.1. [문제] 주사위의 눈을 원하는 값으로 맞추는 로직이 필요하다.....	13
4.2. [난이도] 물리적 흐름 바깥에서 결과를 맞추려는 방식은 모두 한계를 가진다.....	13
4.3. [해결] PD 제어 방식을 활용한 토크 설정	16
4.4. [결과] 토크를 통해 목표 자세에 도달할 수 있다.....	18
4.5. [트레이드오프] 안정적인 도달을 얻을 수는 있지만 확정적인 결과를 얻지는 못한다.....	19
5. 결과 판정을 서버로 옮기며 클라이언트 코드 유지하기	21
5.1. [문제] 원래 게임플레이 코드 전체는 클라이언트에 존재했다.....	21
5.2. [해결] 결과에 영향을 주는 구간만 서버의 역할로 옮긴다.....	21
5.3. [결과] 클라이언트 코드를 일부 유지한 채 결과 판정을 서버로 옮겼다.....	25

5.4. [트레이드오프] 서버로 옮기는 범위를 줄인 대신 그에 대한 비용이 있다.....	25
6. 결과 및 회고.....	27

1. 프로젝트 개요

1.1. 개발 배경

My-YachtDice 프로젝트에서는 여러 주사위가 낙하하고 서로 충돌하며 굴러가는 움직임을 물리 시뮬레이션으로 구현해야 했습니다. 이러한 낙하와 충돌 자체는 기존 물리 엔진을 사용해서도 구현할 수 있습니다.



그림 1. My-YachtDice 프로젝트 플레이 화면

하지만 게임에서는 단순히 주사위를 굴리는 것뿐만 아니라, 원하는 결과에 맞게 주사위를 특정 자세로 유도하는 로직이 필요했습니다. 이 때 주사위의 회전을 직접 설정하는 방식은 물리 연산이 갱신하고 있는 값을 외부에서 덮어쓰는 것이므로, 물체가 굴러가던 흐름과 이어지지 않는 문제가 있었습니다. **목표 자세를 만들어내면서도 진행 중인 물리 운동을 유지할 수 있는 방식이 필요했습니다.**

이를 별도의 게임 로직에서 회전값을 보정하는 방식으로 처리하기보다, 목표 자세 역시 힘과 토크를 통해 물리 연산에 반영할 수 있도록 구성하고자 했습니다. 그래서 **충돌 감지, 접촉 해결, 위치/회전 갱신까지 이어지는 물리엔진 파이프라인**을 추가적으로 구현했습니다. 이후 멀티플레이 환경을 구성하면서 물리 연산을 서버로 옮기고, 각 클라이언트가 동일한 결과를 볼 수 있도록 했습니다.

1.2. 주요 개발 항목 및 핵심 과제

1) 물리 엔진 파이프라인 설계

물체의 힘과 회전 운동부터 충돌 감지, Contact 해결, 위치/회전 갱신까지 이어지는 물리 처리 구조를 구성했습니다. 각 단계의 역할을 분리하고, 고정된 물리 프레임에서 순차적으로 처리할 수 있도록 Physics World를 설계했습니다.

2) 접촉 해결의 반복 수행과 반발 유지

접촉 해결을 한 번만 수행할 경우 처리되지 못한 속도가 프레임마다 누적되어 물체가 접촉면을 파고드는 문제가 있었습니다. 이를 한 프레임에 접촉 해결을 여러 번 반복하는 방식으로 처리했으며, 반복 과정에서 상대 속도가 0으로 수렴하며 반발이 사라지는 문제는 반복 이전의 속도를 별도로 저장해 반발 계산에 적용하는 방식으로 해결했습니다.

3) Orientation Injection

원하는 주사위 결과(눈)를 만들기 위해 Transform의 회전값을 직접 변경하는 것이 아닌, 현재 자세와 목표 자세의 차이를 기준으로 Torque(돌림힘)를 적용하는 내용을 구성했습니다. 이를 통해 기존의 충돌과 회전 운동을 유지하면서 주사위를 목표 Orientation으로 유도했습니다.

4) 서버 권위 물리 연산과 클라이언트 동기화

멀티플레이 환경에서 굴리기 결과가 조작되지 않도록 결과를 결정하는 물리 연산을 서버로 옮겼습니다. 이 때 클라이언트에 이미 작성된 게임플레이 코드의 수정 범위를 줄이기 위해 결과에 영향을 주는 구간만 서버가 담당할 수 있도록 경계를 나누고 이후 구간은 서버의 스냅샷을 각 클라이언트가 보간하여 표시하도록 구성했습니다.

1.3. 프로젝트 정보

구분	내용
프로젝트	MyYachtDice 프로젝트
개발 기간	2026.03 - 2026.04
개발 환경	C++, DirectX11, HLSL, 소켓서버(TCP)
팀 구성	개인 프로젝트

2. 물리 파이프라인 설명 (배경지식 설명)

이 절의 목적은 물리/수학을 설명하는 것이 아니라 이후 3장과 4장을 읽기 위한 지도를 제공하는 것입니다. 이를 위해 물리 엔진의 전체 흐름을 소개합니다.

2.1. 물리 엔진 전체 처리 흐름

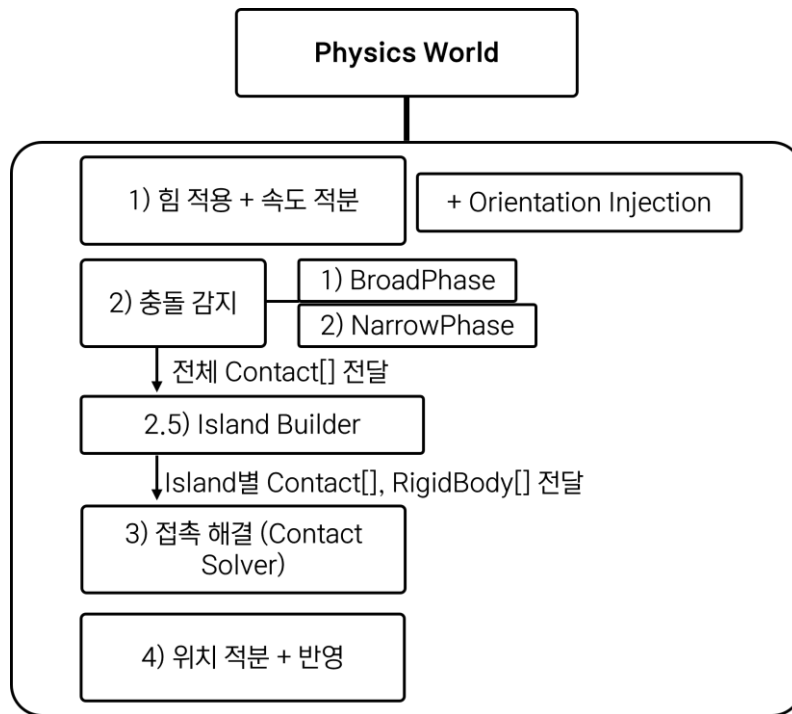


그림 2 Physics World 물리 프레임 처리 흐름

물리 엔진의 전체 흐름은 Physics World라는 환경 내에서 진행됩니다. Physics World는 게임의 일반적인 Update와 분리하여 고정된 시간 간격(1/60s)을 기준으로 갱신하도록 구성했습니다. 한 번의 물리 프레임에서는 현재 물체가 가지고 있는 힘 적용, 속도 적분(1)을 거치고, 물체 간의 충돌(2)을 검사한 뒤, Contact 정보를 구성합니다. 이후 Contact Solver로 전달해 충돌(물체끼리 접촉된 상황)을 해결(3)하고, 계산된 속도와 각속도를 기준으로 물체의 위치와 회전을 갱신(4)합니다.

각 단계는 다음과 같은 역할을 진행합니다.

1) 힘 적용과 속도 적분

Physics World가 아닌 게임 로직 내에서 적용된 힘을 가속도로 전환하여 속도에 누적하고, 중력을 적용할 객체(Rigidbody)는 추가적으로 속도를 누적합니다. 이 과정에서 Orientation Injection(목표 방향으로 자동적으로 회전하게 하는 방식) 연산도 이뤄집니다.

2) 충돌 감지

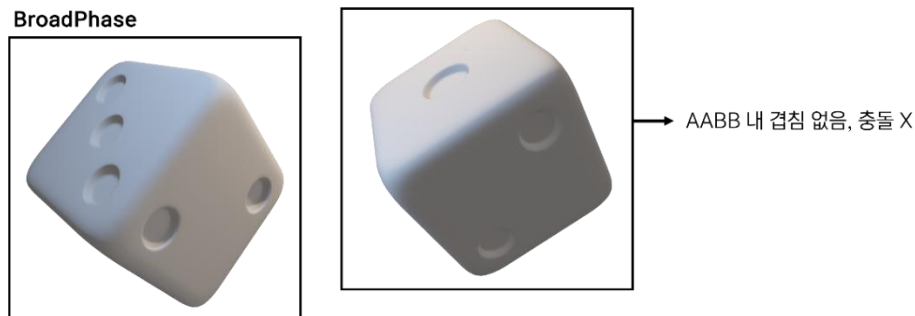


그림 3. BroadPhase 방식

현재 물체들의 위치, 충돌체 정보를 기준으로 서로 충돌하고 있는 대상을 검사합니다. BroadPhase에서는 AABB를 이용하여 충돌하지 않는 충돌체를 걸러냅니다. NarrowPhase에서는 물체의 충돌체 정보를 기반으로 정확한 충돌을 확인합니다. 충돌이 확인되면 이후 단계인 접촉 해결에 필요한 정보(Contact)를 구성합니다.

3) Contact 해결 (Contact Solver)

Contact에는 어떤 충돌체 두 개가 서로 충돌했는지, 어느 지점에서 충돌했는지, 충돌 방향은 어떻게 되는지를 가지고 있습니다. 이를 기준으로 충돌한 물체의 운동 상태(속도, 각속도 등)를 수정하게 됩니다. 이 단계에서 충돌 시 발생하는 반발/마찰/관통을 처리합니다. 이 과정에서 물체의 속도 및 각속도가 완전히 바뀌게 될 수 있습니다.

4) 위치와 회전 갱신

충돌 해결까지 반영된 최종 속도 및 각속도를 물체의 위치와 회전에 갱신합니다.

3절은 Contact solver를 개발하면서 마주했던 문제들을 어떻게 해결했는지를 다루며, 4절은 프로젝트에서 원했던 내용인 Orientation Injection은 어떻게 이루어지는지를 다룹니다.

3. 접촉 해결을 안정적으로 처리하는 Contact Solver 만들기

3.1. [문제] 물체끼리 접촉할 때 접촉면을 파고드는 문제가 있다.

처음 물리 엔진 개발에서 두 물체가 충돌할 때 반발, 마찰, 관통 3요소에 대해 보정을 계산하는 **Contact Solver**를 구현했습니다. Contact Solver를 통해서 두 물체가 충돌했을 시에 물리적 수식을 통해서 물체가 튕겨나가거나 멈추는 동작을 기대했습니다.



그림 4. 접촉 해결이 제대로 진행되지 않아 화면을 파고드는 상황

하지만 구현 이후 실제로 결과를 확인해보니 관통 및 반발 등의 문제가 제대로 해결되지 않아 그림과 같이 바닥면을 파고들거나 물체가 파고든 상태에서 움직이는 등의 동작을 확인하였습니다.

3.2. [난이도] 화면에 보이는 것은 결과뿐, 어느 부분이 원인인지 특정하기 어렵다.

물체가 파고드는 현상은 확인했지만 그 안에서 어느 부분이 잘못되었는지 그리고 어떻게 고쳐야 하는지를 특정하기는 어려웠습니다.

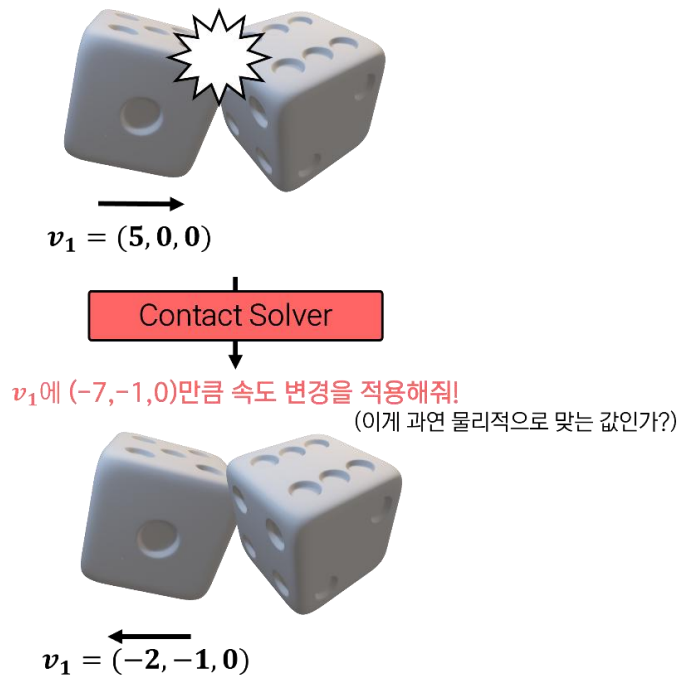


그림 5. Contact Solver의 해결이 물리적으로 맞는지 검증하기 어려운 상황

첫 번째 이유는 **계산 결과를 검증할 기준이 없다**는 점입니다. 접촉 해결은 대부분 충격량 (impulse)을 속도에 적용하는 방식으로 동작하며, 그 과정에서 나오는 값은 수치로 확인할 수 있습니다. 그러나 **이렇게 나오게 된 수치가 정말 물리적으로 올바른 값인지를 판단할 방법이 없습니다**. 코드 작성이 잘못된 것인지, 계산은 의도대로 됐지만 결과가 부족한 경우인지 수치만으로 구분할 수가 없습니다.

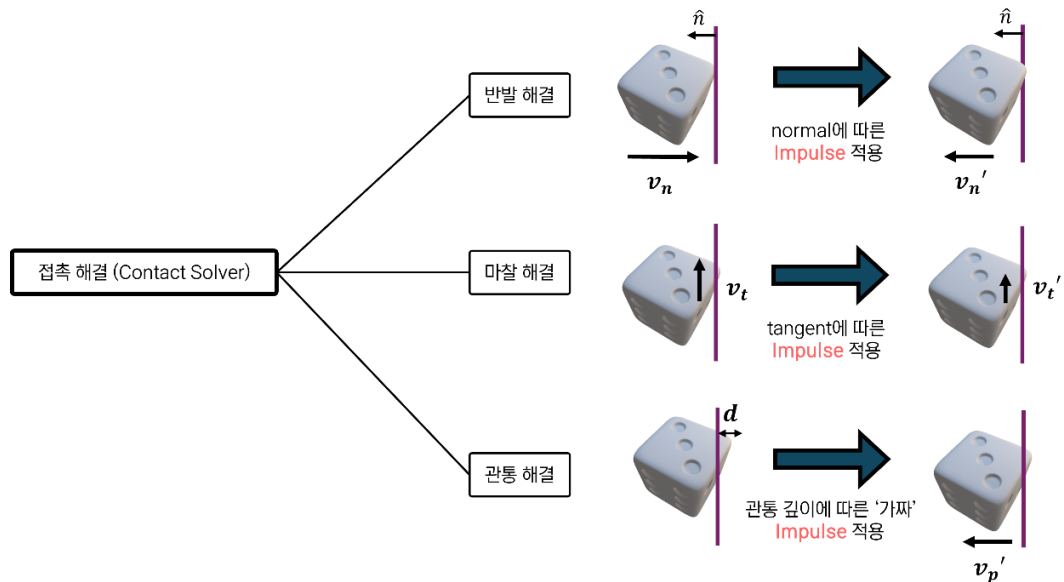


그림 6. 접촉 해결의 세가지 적용 방식

두 번째 이유는 **Contact Solver의 보정 방식이 모두 위치가 아닌 속도를 조정한다**는 점입니다. 반발, 마찰, 관통의 보정이 모두 Rigidbody에 Impulse를 적용하여 속도를 누적하는 방식이고, 접촉한 깊이를 고려해 위치를 조정하는 방식이 아니기 때문에 **누적된 속도 값이 접촉한 깊이를 충분히 해결하지 못한다면 계속해서 파고드는 문제**가 남게 됩니다.

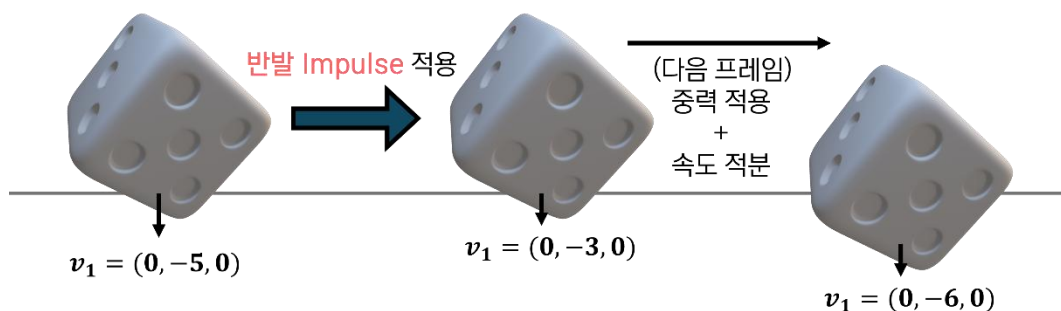


그림 7. 접촉 해결의 impulse 적용이 크게 되지 않는 상황

위의 그림 예시와 같이 적용된 Impulse의 크기가 크지 않아 속도를 충분히 끌어올리지 못한다면, 그 다음 프레임에서 적용된 외력(중력)과 속도에 따른 이동으로 인해 파고들 수 있게 됩니다.

니다. 만약 Impulse의 크기를 키우면 물체가 필요 이상으로 튕겨나가는 문제가 생길 수 있으므로 값을 조정(scaling)하는 방식은 근본적으로 해결할 수 없습니다.

3.3. [해결 1] 접촉 지점에 대한 반복 계산으로 해결한다. (Iterative Solve)

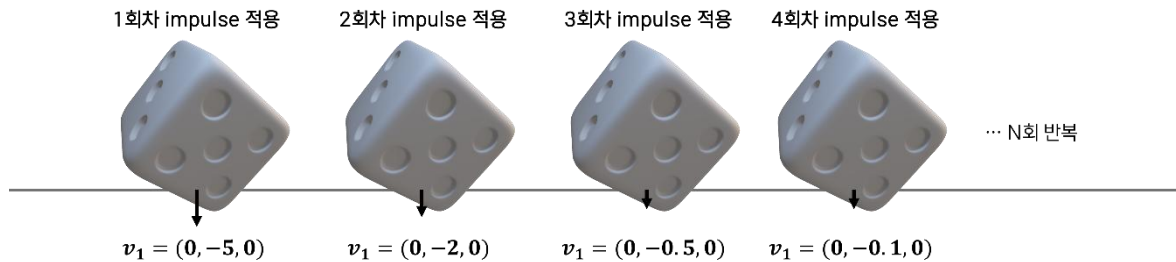


그림 8. 반복 계산 예시

한 번의 적용으로 파고드는 속도가 제거되지 않아, 남은 속도가 사라질 때까지 같은 계산을 반복해서 해결했습니다. 반복할 때마다 갱신된 속도를 기준으로 충격량이 다시 계산되고, 반복이 진행될수록 속도가 0에 가까워지게 됩니다. 그래서 잔여 속도가 없어 다음 프레임에 아래로 계속해서 파고드는 문제가 없어지게 됩니다. 이 프로젝트에서는 총 8회 반복 연산을 하였습니다.

그리고 이 방식은 다중 접촉 지점으로 인한 문제도 안정적으로 해결할 수 있습니다. 접촉을 하나씩 순서대로 해결하면, 앞선 접촉에서 변경된 속도가 다음 접촉의 계산에 반영되어 처음의도와 다른 결과가 나올 수 있습니다.

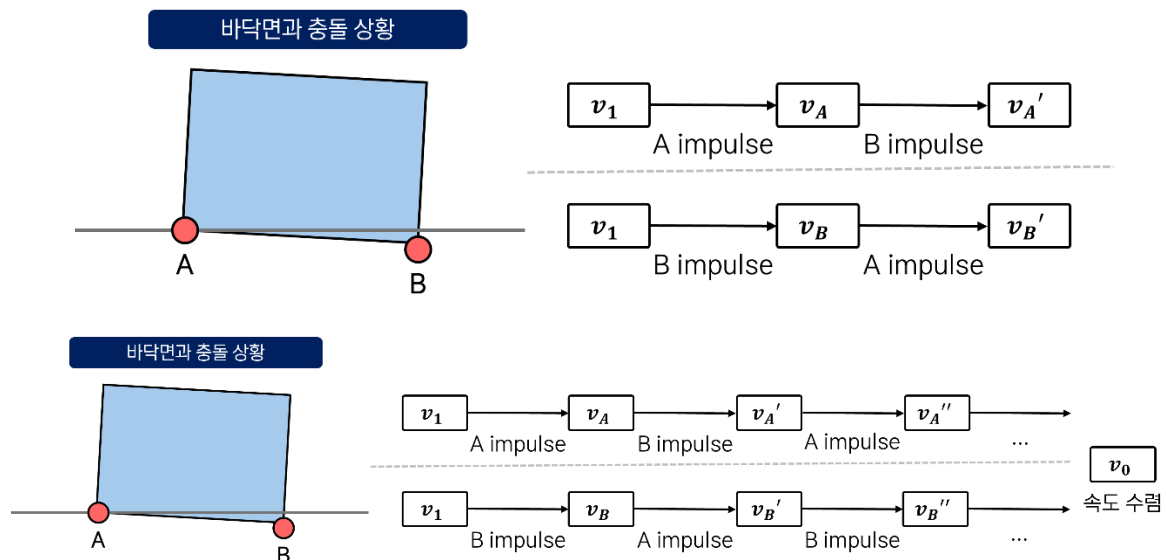


그림 9. (위) 1회 연산 시 속도가 수렴하지 않는 문제상황/(아래) 반복 연산시 속도가 수렴하는 상황

기존에는 접촉 A를 해결하고 바뀐 물체의 속도를 기준으로 접촉 B를 해결하는 것과 반대의 순서로 해결하는 것이 다른 속도를 가질 수도 있습니다. 하지만 반복 해결에서는 모든 접촉을 한 번씩 처리하는 과정을 여러 번 거치므로, **각 접촉의 변화가 서로에게 점진적으로 반영되어 이러한 편차가 줄어들며 속도가 수렴하게 됩니다.**

3.4. [해결 2] 충돌 시점의 속도를 bias로 두어 반발을 유지한다.

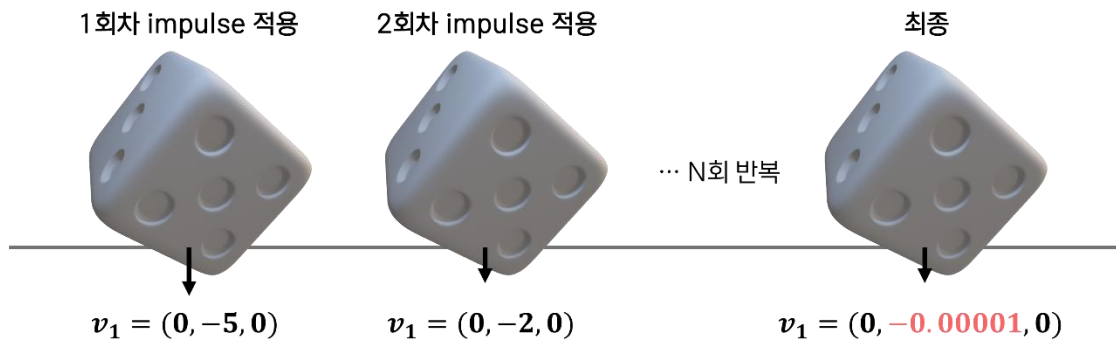
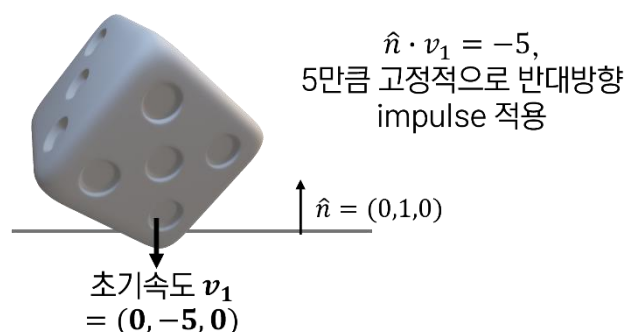


그림 10. 반복연산으로 인해 속도가 0으로 수렴하는 상황

반복 해결을 적용한 이후 물체가 파고드는 현상은 해소되었으나, 충돌 시 튕겨나가는 움직임은 보이지 않았습니다. 그 이유는 **접촉한 물체의 해결 방식이 상대 속도를 0으로 만드는 방향으로 impulse를 적용하기 때문에 정지한 바닥면(물체의 속도가 0)과 부딪혔을 때는 튕겨나가지 않고 곧바로 멈추는 모습을 보여주었습니다.**

속도를 0으로 만드는 것을 해결하기 위해 **반복 해결(iterative solve) 이전 상대 속도를 별도로 저장하여 반발의 impulse의 계산에 더할 수 있도록 사용했습니다.** 이 값을 restitution bias라고 두고 매 반복마다 반발 계산에 함께 적용하면, 상대속도가 0으로 수렴하더라도 처음의 충돌 세기에 해당하는 반발이 유지됩니다.



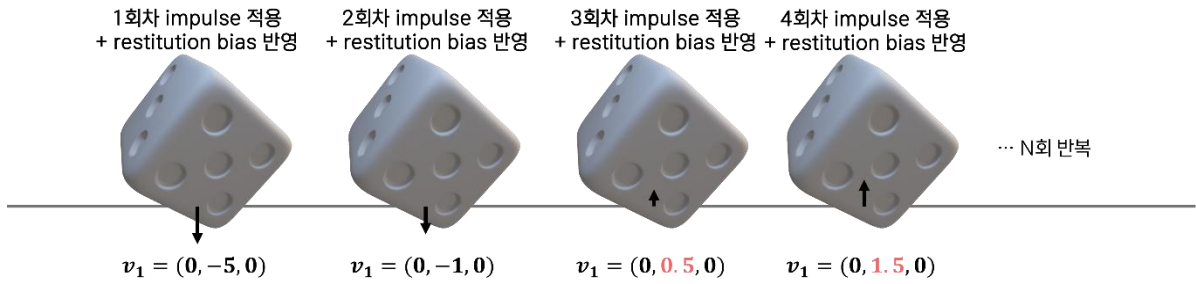


그림 11. bias 적용 예시

위와 같은 예시 상황에서 주사위의 속도와 바닥면 normal을 내적하면 5라는 값이 나옵니다. 반복 연산의 반발에서 5에 해당하는 **impulse**를 추가적으로 계속 적용함으로써 충돌했을 때 멈추는 것보다 반대방향으로 밀려 나게 되는 것처럼 보이게 됩니다.

3.5. [결과] 파고드는 것이 누적되지 않고 반발이 유지된다.

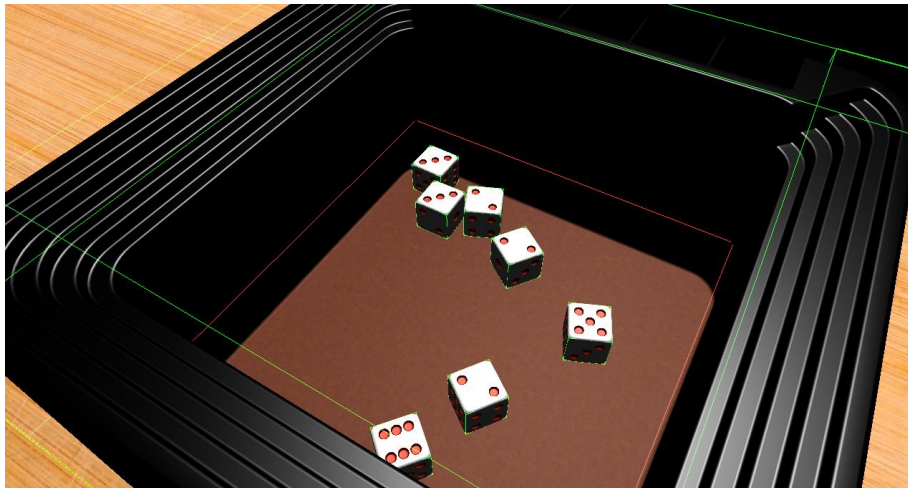


그림 12. 문제 해결 이후 상태

접촉 해결 8회 반복 도입 이후, **물체가 바닥면을 파고드는 현상이 나타나지 않았습니다**. 반복을 적용하지 않았을 때는 접촉이 유지되는 동안 매 프레임 물체가 아래로 가라앉으며 움직이는 현상이 나타났으나, 반복 적용 이후에는 정지한 상태가 유지하게 되었습니다.

Restitution bias를 적용한 이후에는 **물체가 튕겨 나가는 움직임이** 나타났습니다. 반복 해결로 접촉면 방향의 상대 속도가 0에 수렴하더라도, 충돌 시점의 속도를 기준으로 계산된 반발이 반대방향으로 밀어내는 힘으로 작용하기 때문입니다.

이 두 가지 처리를 통해 다수의 주사위가 동시에 충돌하고 쌓이는 상황에서도 안정적으로 접촉을 처리할 수 있게 되었습니다.

3.6. [트레이드오프] 안정적인 접촉 처리를 얻는 대신 연산량과 정확도를 내주었다.

1) 반복 횟수를 늘릴수록 연산량이 함께 증가한다.

반복 연산의 횟수를 늘리면 속도가 좀 더 목표값으로 수렴하게 되지만, 그만큼 반복 횟수 (N번)가 늘어나기 때문에 연산량도 정비례하게 됩니다. 8회는 실험을 통해 확인했을 때 ‘원하는 형태로 물리적 동작이 일어나는가’를 기준으로 정한 값입니다. 이보다 더 줄어들게 된다면 기존 처럼 물체가 파고드는 문제가 생길 수도 있으며, 더 늘어나게 된다면 정밀도는 늘어나나 연산에 대한 부하가 커질 수도 있습니다.

2) Restitution bias는 반복 이전 시점의 값으로 실제 충돌 세기 및 방향과는 다를 수 있다.

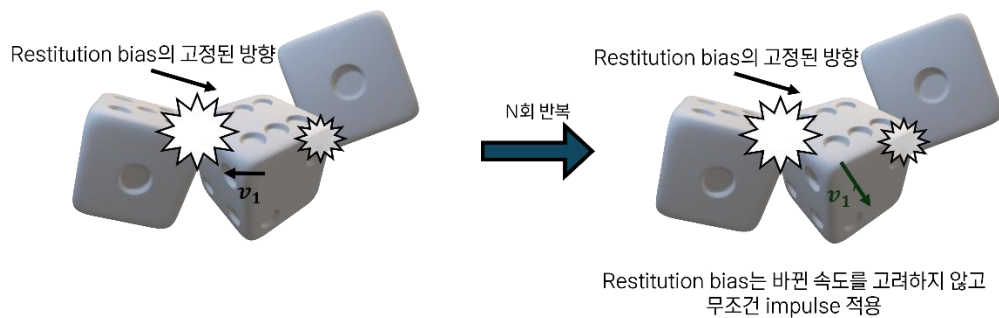


그림 13. 고정된 방향 bias의 문제

Restitution bias는 반복 해결을 시작하기 전에 계산한 상대 속도를 통해 정해집니다. 반복 과정에서 다른 접촉의 처리로 속도가 크게 변해도 restitution bias는 고정되어 있으므로 여러 물체가 동시에 충돌하는 상황에서는 실제 충돌 세기와 차이가 생길 수 있습니다. 또한 반복 해결로 얻은 안정성과 반복 이전 값을 기준으로 반발을 유지하는 방식은 서로 다른 시점의 값을 사용한다는 점에서 완전히 일치하지는 않습니다.

4. 물리 운동을 유지하며 목표 자세(Orientation)로 유도하기

4.1. [문제] 주사위의 눈을 원하는 값으로 맞추는 로직이 필요하다.

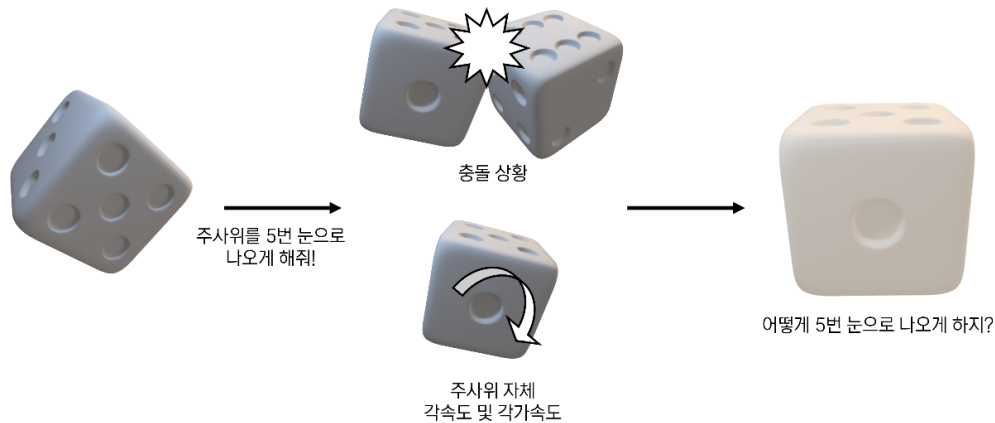


그림 14. 원하는 회전으로 적용하기 위해 고려해야 하는 상황

My-YachtDice에서는 주사위를 굴린 뒤 원하는 눈이 위를 향해 나오도록 주사위를 그 방향으로 설정하는 방법이 필요했습니다. 이 방식을 수행하는 가장 단순한 방법은 주사위의 현재 방향을 원하는 방향으로 나타나도록 SetOrientation을 하는 것입니다. 하지만 이 방식은 현재 주사위가 어느 속도로 이동하고 어느 각속도로 회전하는지에 대한 상황을 고려하지 않고 설정하는 방식이기 때문에 물리적인 흐름과는 맞지 않았습니다. 그래서 목표 방향으로 자연스럽게 향하게 하는 새로운 로직이 필요했습니다.

4.2. [난이도] 물리적 흐름 바깥에서 결과를 맞추려는 방식은 모두 한계를 가진다.

현재의 해결 방식을 적용하기 이전, 목표 방향으로 주사위를 맞추는 것에는 두가지 방식 중 하나만으로 해결이 가능할 것이라고 생각했습니다.

- 1) 원하는 회전 결과가 나올 때까지 시뮬레이션을 돌리고, 시뮬레이션 결과가 조건에 만족하면 초기 설정값(속도/각속도/위치)을 게임에 적용한다.

```

bool IsSimulationCompleted = false; // 시뮬레이션 끝났는지 체크
while (!IsSimulationCompleted)
{
    for (auto& dice : NowDices)
    {
        dice->Throw(); // 모든 주사위 랜덤으로 굴리기 시작
        dice->RecordStat(); // 현재의 위치/속도/가속도를 저장
    }

    for (auto& dice : NowDices)
    {
        int StopDiceCount = 0;
        if (!dice->IsStopped()) // 주사위가 하나라도 멈추지 않았으면
        {
            PhysicsWorld->Step(1.f / 60.f); // 물리 시뮬레이션 계속 실행
            StopDiceCount = 0;
            continue;
        }
        else
        {
            ++StopDiceCount;
            if (NowDices.size() == StopDiceCount) // 다 멈췄다면 시뮬레이션 일시 종료
                break;
        }
    }

    IsSimulationCompleted = true;
    for (int i = 0; i < NowDices.size(); ++i)
    {
        if (NowDices[i]->GetTopFace() != TargetValues[i]) // 현재 주사위의 값이 목표한 값이 아니라면 주사위 다시 굴리기
        {
            IsSimulationCompleted = false;
            break;
        }
    }

    if (IsSimulationCompleted)
    {
        ThrowAllDicesWithNowStat(); // 현재 주사위의 Stat에 맞춰서 주사위를 돌리자!
    }
}

```

그림 15. 목표 값이 나올 때까지 시뮬레이션 돌려보기 (의사코드)

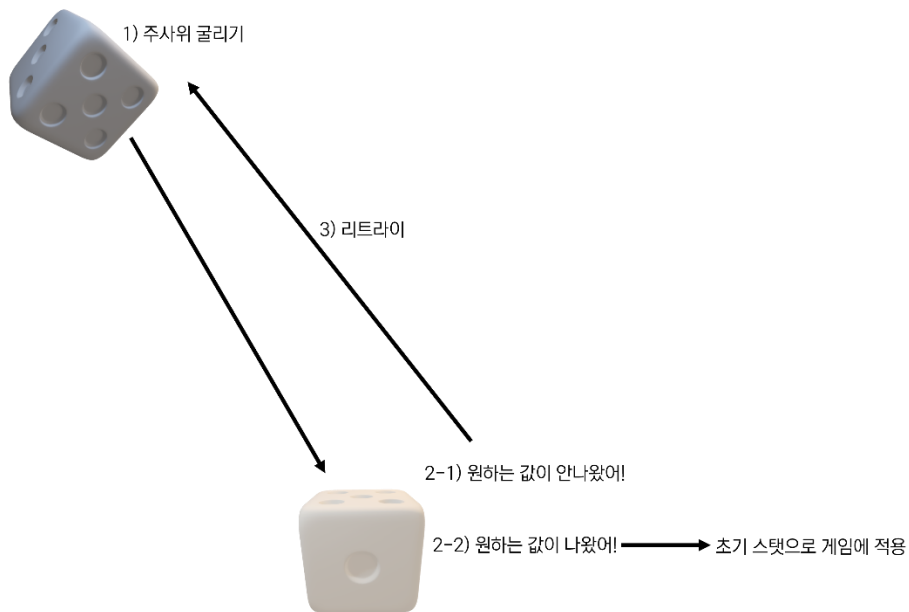
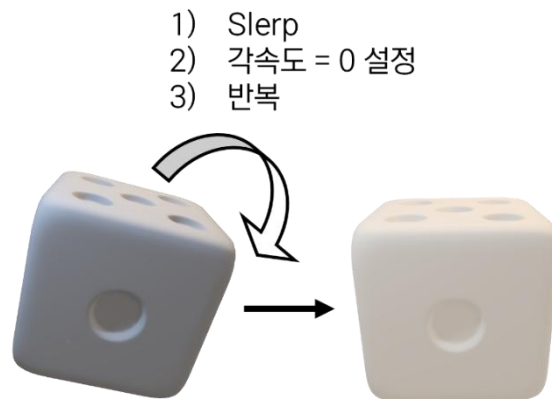


그림 16. 시뮬레이션 방식의 흐름

이 방식은 원하는 값이 나올 때까지 주사위들을 랜덤한 속도, 각속도, 위치를 설정하고 시뮬레이션을 돌려보며 원하는 값이 나오게 된다면 초기값을 재활용하여 게임 로직에 활용하는 방식입니다. 이 방식의 문제점은 원하는 값이 나오지 않게 된다면 무한 시뮬레이션이 일어날 수 있다는 것이며, PhysicsWorld의 Step 함수 자체가 전체적인 물리 연산을 진행하는 것이며 이

방식은 전체 시간(한 프레임이 아닌 주사위가 멈출 때까지 단위 시간 간격으로 반복 연산)에 대한 연산이기 때문에 연산 시간의 증가를 일으킬 수 있다는 점입니다. 또한 시뮬레이션 결과를 통해 얻은 초기값을 실제 플레이에 적용해도 실제와는 다른 값이 나올 수 있다는 가능성이 존재합니다.

- 2) 속도가 많이 줄어들었을 때, 현재 회전과 목표 회전 사이를 보간(Slerp)하고, 각속도를 강제로 제거한다.



이 방식은 주사위의 속도와 각속도가 기준 값보다 작을 경우, 목표 회전 값으로 보간(Slerp)하여 향하며 각속도를 0으로 설정하는 방식입니다. 이 방식으로 진행할 경우 **프로젝트의 물리엔진과 충돌하는 내용이 존재했습니다.**

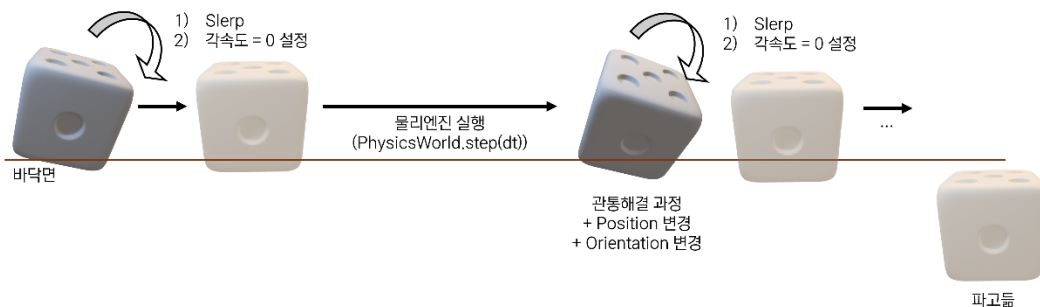


그림 17. 실제 Slerp 방식 적용시 나타나는 문제

물리엔진에 존재하는 Contact Solver는 충돌로 인한 관통을 해결하기 위해 ‘가짜 속도’와 ‘가짜 각속도’를 부여합니다(실질적으로는 가짜 impulse 적용). 이 방식은 실제 물체의 속도와 각속도에 반영이 되지 않지만, 관통을 해결하기 위해 위치를 조절하고 회전을 조절하는 내용으로 사용됩니다. 관통된 깊이가 깊을수록 튀어나가기 위한 가짜 속도와 가짜 각속도의 크기는 커지게 되고 그에 따라 보정된 회전은 원치 않는 방향으로 살짝 더 돌아갈 수 있습니다. **이후 강제 보간(Slerp)을 진행하면 원치 않는 방향에서 방향 재설정**이 이뤄져 관통된 깊이가 이전보다

커지게 됩니다. 최종적으로는 물체가 바닥을 파고드는 문제가 생기게 됩니다.

2번 문제에 따라서 물리엔진의 흐름을 거스르는 방식(각속도를 억지로 0으로 만들고, 보간을 통해 목표 회전으로 설정)으로 설정을 하는 것은 근본적으로 해결할 수 없다는 것을 알았습니다. 그래서 끊기지 않는 회전상태를 유지하기 위해서 **토크(돌림힘, τ)를 추가적으로 부여하는 방식을 고려해야** 했습니다.

- 왜 토크를 부여해야 하는가? 각속도를 추가하는 것은 안되는가?

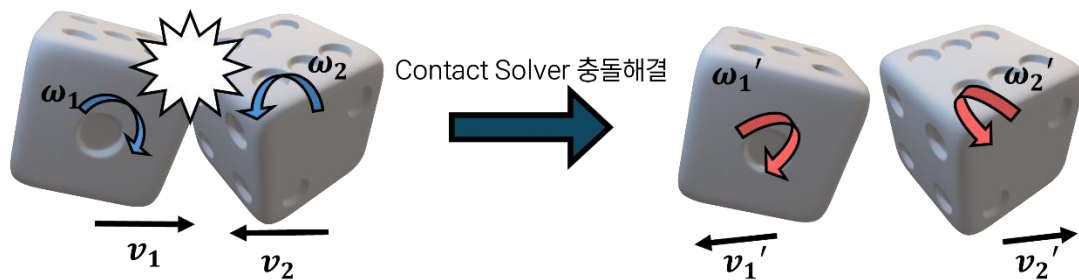


그림 18. 충돌 해결 이후 나타나는 상황(속도 변경, 각속도 변경)

각속도를 추가하는 방식은 주사위끼리 혹은 벽과 부딪힐 때 문제가 있습니다. **충돌을 하게 되면 충돌 전 속도 및 각속도($v_1, v_2, \omega_1, \omega_2$)가 Contact solver의 반발 및 마찰 해결 단계에 의해 완전히 바뀌게 됩니다($v_1', v_2', \omega_1', \omega_2'$)**. 이러한 이유로 조정된 각속도는 충돌 시에 완전히 무효화가 되기 때문에 현재의 상황을 계속 반영할 수 있는 토크를 추가하는 것으로 해결하였습니다.

4.3. [해결] PD 제어 방식을 활용한 토크 설정

주사위에 더할 토크를 계산할 방식으로 제어공학에 존재하는 개념인 PD제어를 사용했습니다.

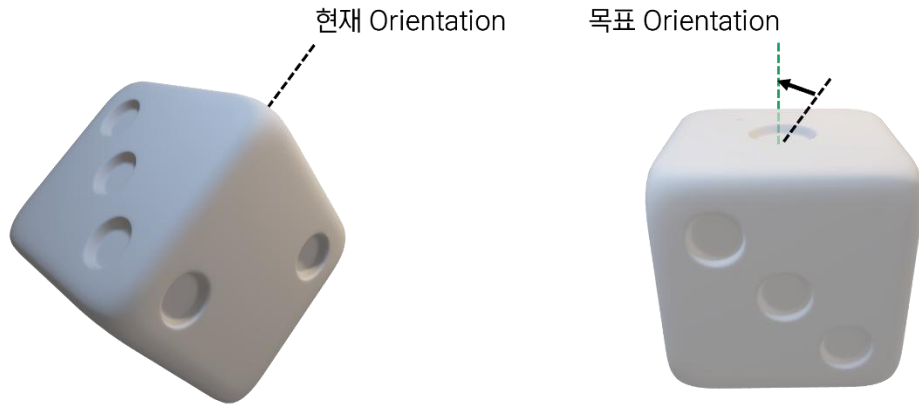


그림 19. 현재 Orientation과 목표 Orientation의 회전 오차

1) P 제어 방식 (회전 오차에 비례하여 토크를 적용하는 방식, 1차 적용)

$$\tau = K_p \cdot \theta_{err} \cdot \hat{n}_{err}$$

(K_p : 비례 이득 상수, $\hat{n}_{err}$, θ_{err} : 오차에 의한 회전 축과 각도)

처음에는 목표 Orientation으로 주사위를 유도하기 위해 먼저 현재 자세와 목표 자세의 차이에 비례하여 토크를 적용하는 P 제어 방식을 사용했습니다. 목표 회전과의 차이가 클수록 더 큰 토크를 적용하고, 가까워질수록 토크가 작아지도록 구성했습니다.

하지만 P 제어만 적용했을 때는 목표 자세에 수렴하더라도 기존의 각속도가 남아 있어 목표를 지나치고 반대 방향으로 크게 회전하는 상황이 지속되어 진동을 하는 듯한 감각이 남아있었습니다. P 제어는 현재 자세의 오차만을 기준으로 제어하기 때문에, 물체가 얼마나 빠르게 회전하고 있는지 고려하지 못하는 것이 진동을 하는 원인이었습니다.

2) PD 제어 방식 (실제 적용한 방식)

$$\tau = K_p \cdot \theta_{err} \cdot \hat{n}_{err} - K_d \cdot \omega$$

(K_d : 감쇠 상수, ω : 현재 각속도)

```

void RigidBody::ApplyOrientationInjection()
{
    if (!hasOrientationTarget)
        return;

    // 목표 회전, 현재 회전의 오차
    Quat q_error = orientationTarget * QuatConjugate(orientation);

    if (0.f > q_error.w)
        q_error = Quat(-q_error.x, -q_error.y, -q_error.z, -q_error.w);

    Vec3 outAxis;
    float outAngle;

    QuatToAxisAngle(q_error, outAxis, outAngle);

    // outAxis : 오차에 의한 축
    // outAngle : 오차 각도
    // orientationInjectionKp : 비례 이득 상수
    // angular velocity : 현재 각속도
    // orientationInjectionKd : 감쇠 상수
    Vec3 torque = outAxis * (outAngle * orientationInjectionKp) -
        angularVelocity * orientationInjectionKd;

    // Torque 적용
    torqueAccum += torque;
}

```

그림 20. PD 제어 방식 코드

P 제어 방식을 보완하기 위해 PD 제어 방식을 최종적으로 적용했습니다. PD 제어는 P항에 더해, 현재 변화량을 억제하는 D항을 함께 사용하는 방식입니다. 여기서 D항은 현재 각속도에 해당하는 값으로, 빠르게 물체가 회전하고 있다면 이를 감쇠할 수 있는 토크를 추가적으로 구성했습니다.

이를 통해 목표 자세로 회전하는 힘은 유지하면서 목표에 가까워졌을 때 남아 있는 회전 운동을 줄여, P 제어에서 발생했던 진동을 완화하고 목표 Orientation으로 안정적으로 수렴하도록 했습니다.

4.4. [결과] 토크를 통해 목표 자세에 도달할 수 있다.

여러 번 주사위 굴리기를 시도하여 P제어와 PD 제어 방식을 수렴 확률과 수렴 시간 데이터를 통해 비교해 보았습니다. 측정은 속도가 0.2이하에 들어올 때, 목표하는 주사위 눈과 현재 주사위의 눈을 비교하여 성공과 실패 여부에 따른 기록 통해 수렴 확률을 계산하였고, 수렴 시간은 속도가 4이하에 들어온 순간부터 0.2이하에 들어온 순간까지 몇 초의 시간이 걸렸는가를 통해 값을 기록하였습니다.

표 1. 기본/P 제어/PD 제어 수치 비교

	기본 굴리기	P 제어 ($K_p = 40$)	PD 제어 ($K_p = 40, K_d =$
--	--------	---------------------	---------------------------

			4)
수렴 확률	16.66% (1/6)	99.0% (104번 중 103번 성공)	99.1% (111번 중 110번 성공)
수렴 시간	평균 1.10초	평균 1.258초	평균 0.59초

P 제어와 PD 제어 두 방식 모두 99% 확률로 목표자세에 도달할 수 있었습니다. 회전값을 직접 Set 하거나 인위적인 보간 없이 추가적인 토크를 적용하는 것으로 원하는 눈이 위를 향하도록 만들 수 있었습니다.

P 제어와 PD 제어에서 수렴 확률에 대해서는 차이가 없었습니다. 하지만 **PD 제어 방식이 P 제어나 기본으로 주사위를 굴리는 것에 비해 수렴시간이 0.6초가량 짧은 것**을 확인할 수 있었습니다. PD제어의 D항이 현재 각속도의 감쇠로 작동할 수 있기 때문에 주사위가 회전하는 것을 멈추는데 많이 관여함으로써 수렴시간이 감소한 것입니다.

이 시간 차이는 플레이어가 굴리기 시도 후 대기하는 시간과 비례합니다. 멀티플레이 환경에서는 **늘어지는 플레이 타임을 줄일 수 있다**는 점과 더불어서 **동기화를 위한 패킷(주사위 위치/회전 스냅샷)**을 줄일 수 있다는 기대가 있다는 것을 판단할 수 있습니다.

4.5. [트레이드오프] 안정적인 도달을 얻을 수는 있지만 확정적인 결과를 얻지는 못한다.

PD 제어 방식을 통해서 원하는 주사위 눈을 얻는 방식을 마련할 수 있었고, 수렴 시간이 줄어드는 이점 또한 있었습니다. 하지만 이에 대한 트레이드 오프도 존재합니다.

1) 원하는 주사위의 눈을 100% 얻을 수는 없다. (결과 보장이 없다)

결과에서 보듯이 확정적으로 원하는 눈을 얻을 수는 없었습니다. 토크는 현재 상태를 고려하여 적용되기 때문에 충돌에 의해 순간적으로 정지하게 된다면 목표 자세로 돌아가려는 힘의 부족으로 원하는 값으로 설정되지 않을 수 있습니다.

2) K_p 와 K_d 는 직접 튜닝해야 한다. 튜닝을 하더라도 질량(mass)에 따라서 결과를 보장할 수 없다.

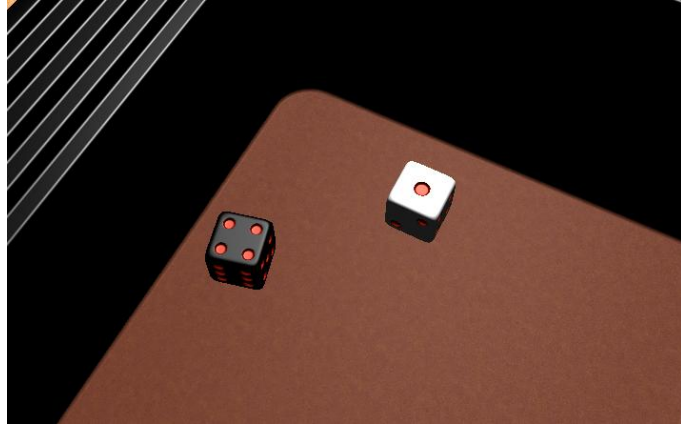


그림 21. 다른 질량을 사용할 때 원하는 눈이 나오지 않는 상황(주사위 값 1 목표)

목표한 눈 설정을 위해서는 제어식에 사용되는 상수를 바꿔가며 적절한 값을 찾아 설정해야 합니다. 이 값은 Rigidbody의 질량에 의해서도 바뀔 수 있습니다. 돌림힘을 각가속도로 전환하는 과정에서 Rigidbody의 질량(정확히는 관성 텐서)를 고려하여 계산되기 때문에 큰 질량으로 바뀔 경우에는 적용되지 않을 수 있습니다.

3) 주사위가 한 곳에 뭉치면 결과를 보장할 수 없다.

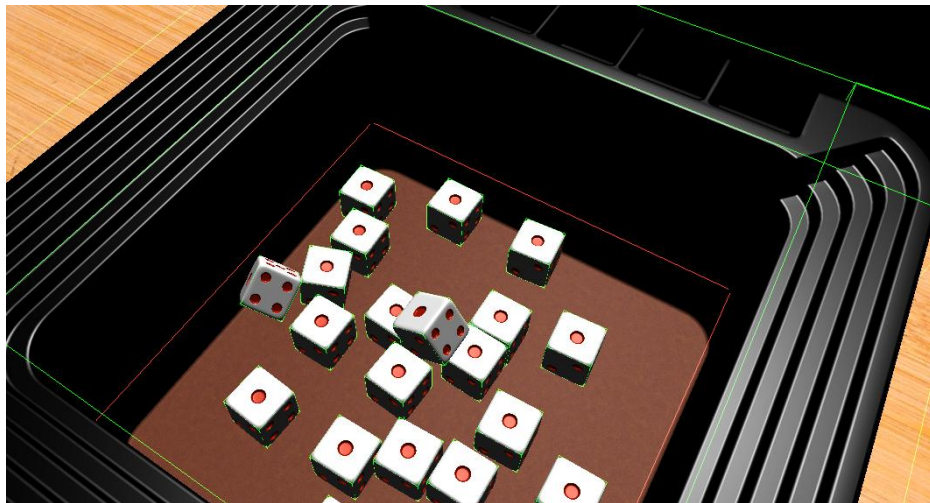


그림 22. 매우 많은 물체가 있을 때 겹쳐져서 원하는 값이 나오지 않는 상황

만약 여러 주사위들이 한 구역에 겹쳐지게 되는 경우에는 인접한 영역에 존재하는 주사위들로 인해 추가되는 돌림힘이 각속도 변경에 반영되지 않을 수 있습니다.

5. 결과 판정을 서버로 옮기며 클라이언트 코드 유지하기

5.1. [문제] 원래 게임플레이 코드 전체는 클라이언트에 존재했다.

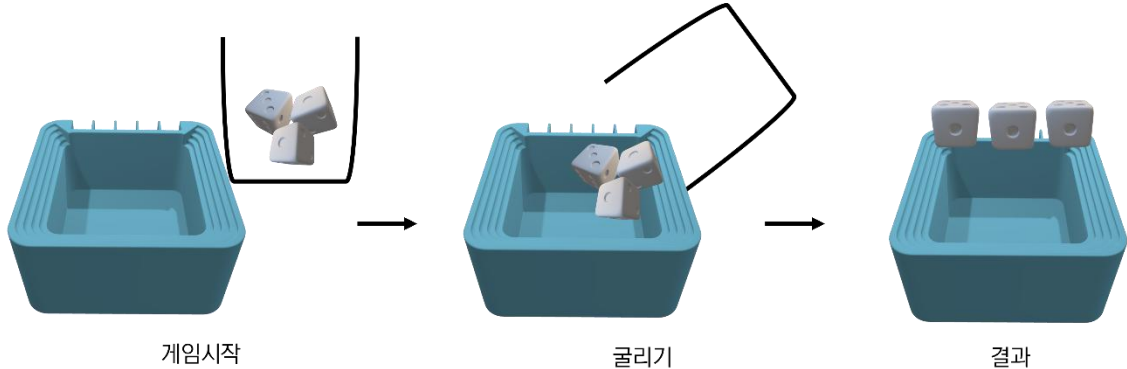


그림 23. 게임 전체 흐름

처음에는 직접 만든 물리엔진에 대한 환경 테스트를 진행하면서 게임 플레이에 대한 코드 작성도 클라이언트에서 진행했습니다. 주사위를 굴리고 점수가 판단되는 시점까지 **결과를 얻는 과정 전체가 클라이언트 안에서 결정되는** 구조였습니다.

이후 멀티플레이 환경을 구성하는 과정에서 이 구조의 문제가 드러났습니다. 야추는 주사위의 눈이 곧 점수가 되므로 굴리기 결과가 게임의 승패를 직접 결정합니다. 하지만 이 **결과를 만들어내는 과정 자체가 클라이언트에 있으면 플레이어가 임의의 결과를 만들어낼 수도 있는 상태**였습니다. 따라서 결과를 결정하는 과정을 서버에서 수행될 수 있도록 옮기고, 서버에서 결과를 각 클라이언트가 동일하게 보도록 만드는 작업이 필요했습니다.

5.2. [해결] 결과에 영향을 주는 구간만 서버의 역할로 옮긴다.

1) 클라이언트가 동작하는 구간과 서버가 동작하는 구간을 나눈다.

클라이언트의 환경 변경을 최소화하면서 서버의 권위로 옮기기 위해 **클라이언트에서 동작하는 구간과 서버에서 동작하는 구간을 나누었습니다.** 여기서 클라이언트에서 동작해도 상관이 없다는 기준은 클라이언트에서 물리 엔진을 실행해도 결과에 영향이 없다는 기준으로 정했습니다. 컵 안에서의 물리적 실행은 이후 서버가 주사위 위치를 재설정하는 시점에 모두 폐기되므로, 이 구간이 클라이언트에서 동작하더라도 최종결과에는 반영되지 않습니다.

서버가 물리 연산을 이어받으려면 **주사위의 상태(위치/속도)를 서버 기준으로 다시 설정해야 합니다.** 이 값은 직전까지 클라이언트에서 갱신되던 위치와 이어지지 않으므로, 주사위가 화면에 보이는 상태에서 재설정하면 순간적으로 이동한 것처럼 나타나게 됩니다.

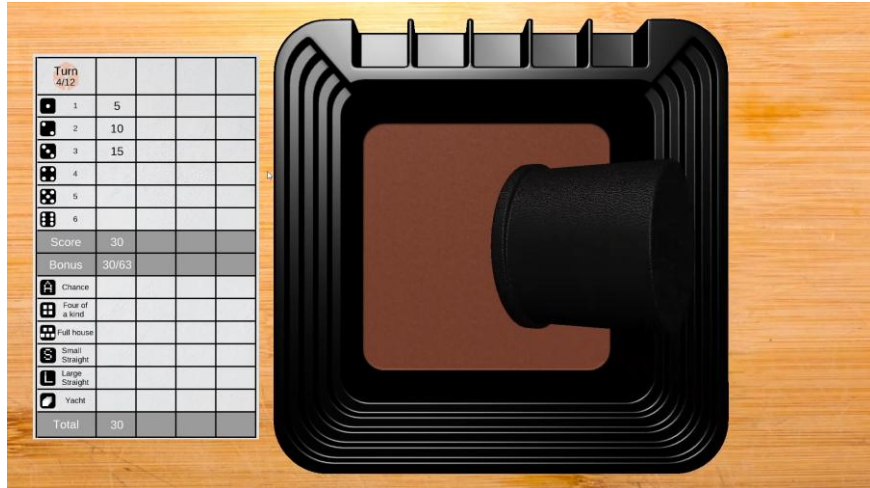


그림 24. 컵이 뒤집히는 시점, 주사위가 보이지 않는 상황

컵이 뒤집히는 시점에는 주사위가 컵에 가려져 카메라에 보이지 않게 됩니다. 주사위의 위치를 새로 설정해야 한다는 지점과 화면에 드러나지 않는다는 지점이 일치하여, 클라이언트의 입력을 받고 컵이 뒤집히는 시점 (C_THROW_DICE 메시지 수신 시점)을 서버 권위로 전환하는 지점으로 정했습니다.

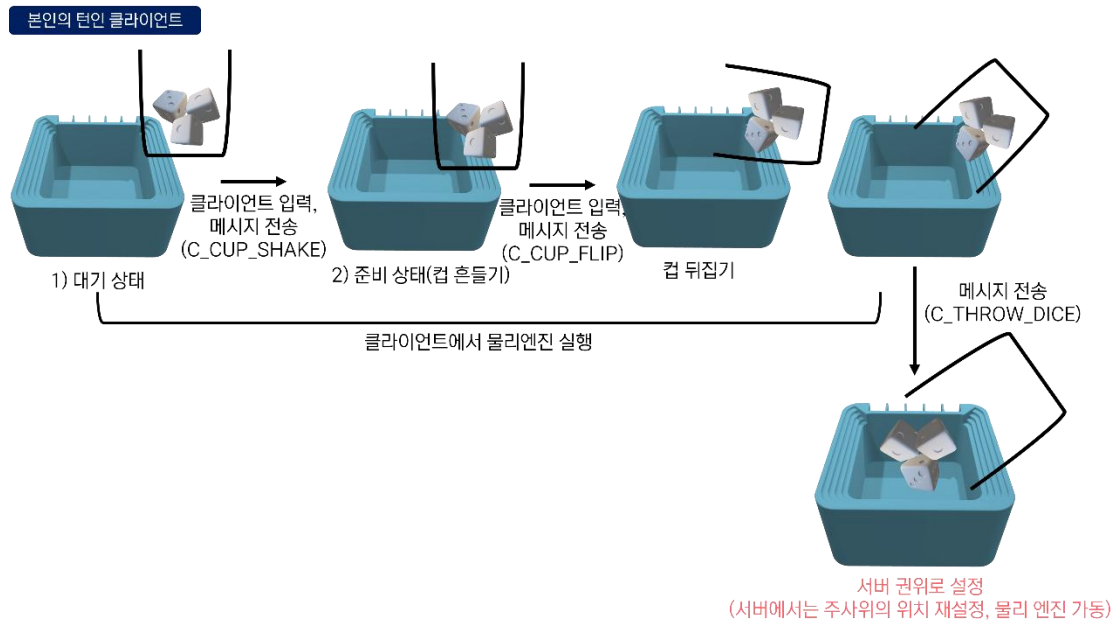


그림 25. 본인의 턴인 클라이언트 상태 흐름 및 패킷 흐름

해당 시점에는 서버는 주사위의 위치와 속도 및 각속도를 랜덤으로 재설정하고 물리 엔진을 가동합니다. 이 시점 이후부터 서버는 각 클라이언트에게 주사위의 스냅샷을 주기적으로 보내게 됩니다. 이 시점부터 모든 클라이언트는 서버의 스냅샷에 따라서 주사위의 위치 및 회전을 보간하며 설정하게 됩니다.

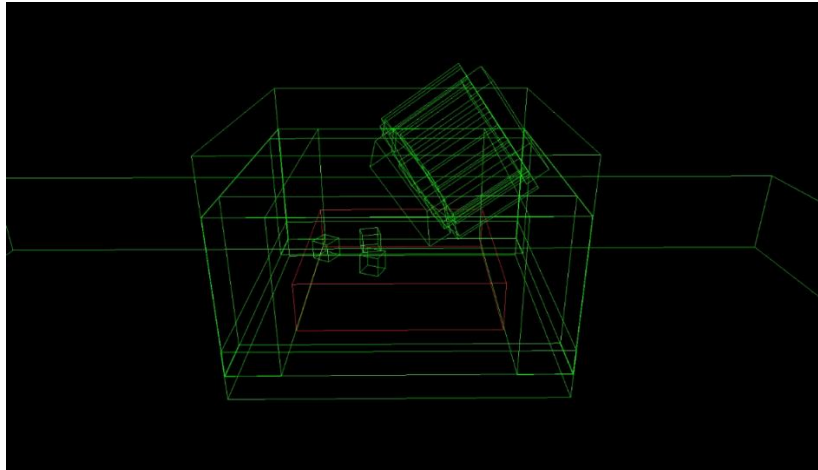


그림 26. 서버에서 구성된 물리 환경 시각화

서버는 주사위의 결과만 얻으면 되기 때문에 고정된 물리적 환경을 구성하도록 했습니다. 이는 서버는 클라이언트의 연출 상태를 알 필요가 없기 때문입니다. 그림과 같이 컵의 콜라이더를 고정된 위치에 두고, 서버 권위의 시점이 되면 콜라이더 안의 위치에서 주사위의 위치를 재설정하게 됩니다.

- 그렇다면 자신의 턴이 아닌 클라이언트는 어떻게 동작하는가?

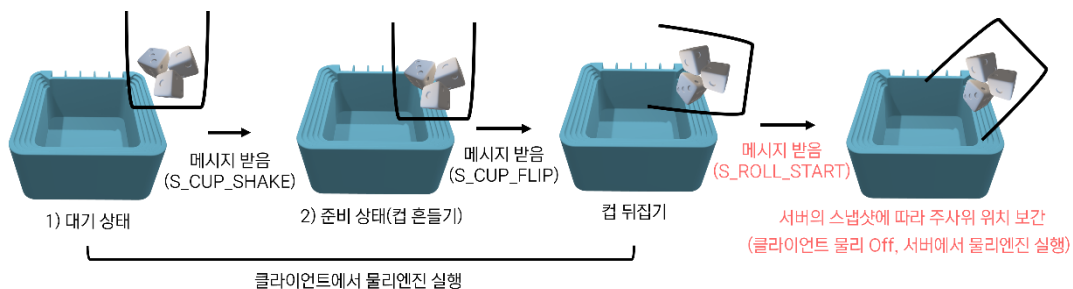


그림 27. 자신의 턴이 아닌 클라이언트의 패킷 흐름

자신의 턴이 아닌 클라이언트는 그림과 같은 흐름으로 동작합니다. 각 클라이언트는 자신의 물리엔진을 가동하면서 주사위의 위치를 설정하고, 메시지를 받을 때마다 메시지에 맞는 동작을 수행하게 됩니다. 예를 들어, **S_CUP_SHAKE**라는 메시지를 받으면 ‘**컵을 흔들기 시작**’하고 컵에 있는 주사위는 클라이언트의 물리엔진에 따라 위치가 설정되게 됩니다. 서버 권위로 주사위를 설정하겠다는 메시지(**S_ROLL_START**)를 받은 이후부터 서버의 스냅샷에 따라 위치를 보간하며 설정하게 됩니다.

2) 주사위의 위치 동기화 (스냅샷 보간 방식)

서버 권위 구간에서 클라이언트로 전송하는 스냅샷에는 주사위의 위치(**Vector3**)와 회전

(Quaternion)만 포함했습니다. 클라이언트는 수신한 값을 이전 값과 보간하여 화면에 반영하며, 이 구간동안 클라이언트의 물리 연산은 주사위에 영향을 주지 않습니다. 위치는 선형 보간(Lerp), 회전은 구면 선형 보간(Slerp)을 사용했습니다.

속도와 각속도를 포함하지 않은 것은 클라이언트가 주사위의 다음 상태를 예측할 필요가 없기 때문입니다(Dead Reckoning이 굳이 필요하지 않습니다). 속도에 따라 서버의 응답을 기다리는 동안 자체적으로 다음 상태를 계산해야 하거나 즉시 반응을 해야 하는 실시간 요소가 존재하지 않기 때문에 서버가 보내주는 값 대로 반영하는 것이 충분했습니다.

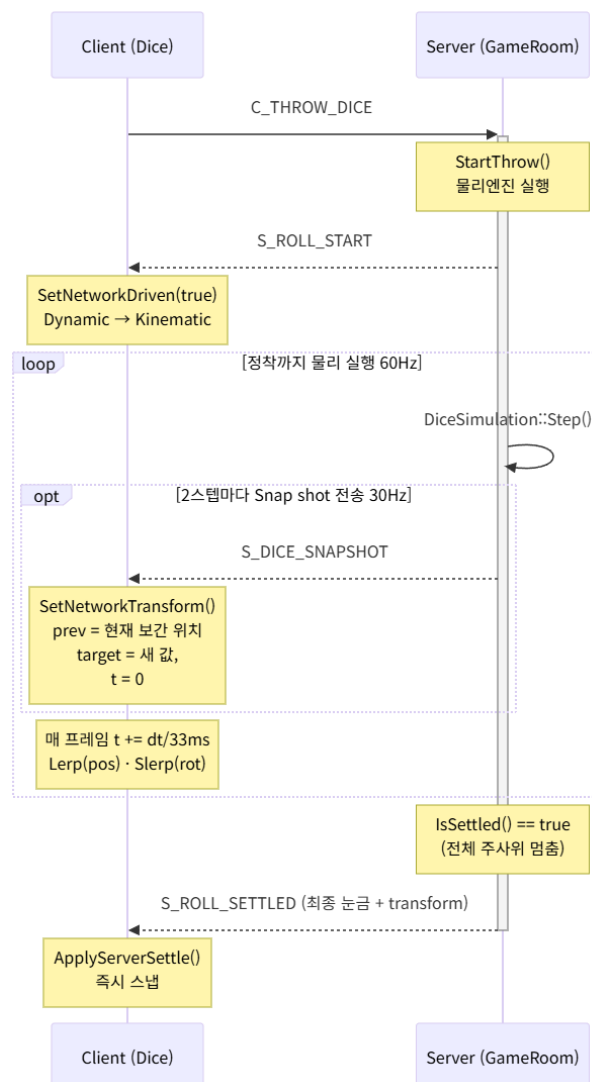


그림 28. 클라이언트와 서버간 주사위 위치/회전 동기화 흐름

서버에서 물리 가동시작부터 주사위의 값이 결정되기까지 흐름은 위의 다이어그램과 같습니다. 서버는 1/30초 간격으로 스냅샷을 전송하면 클라이언트는 해당 스냅샷을 target으로 저장

하고 현재의 정보를 prev에 저장하며, 경과시간을 0으로 설정합니다. 그리고 매 업데이트 루프마다 prev와 target 사이를 lerp하며 주사위의 위치 및 회전을 설정하게 됩니다.

5.3. [결과] 클라이언트 코드를 일부 유지한 채 결과 판정을 서버로 옮겼다.

결과를 결정하는 물리 연산은 서버에서만 수행되고, 클라이언트는 서버가 전송한 스냅샷을 표시하는 역할만 담당하게 되었습니다. 클라이언트에서 주사위의 상태를 변경하더라도 서버가 계산한 결과에는 반영되지 않아 굴리기 결과를 조작할 수 없습니다.

클라이언트에서 서버로 옮긴 범위는 **C_THROW_DICE** 메시지 이후의 물리 연산과 결과 판정으로 한정되었습니다. 그 이전 구간인 주사위 굴리기 입력 처리와 컵 흔들기 연출, 컵 안에서 물리 연산은 기존 클라이언트 코드를 그대로 사용할 수 있었습니다. 결과에 영향을 주는 구간만 서버로 옮긴다는 기준에 따라, 작성되어 있던 게임플레이 코드의 수정 범위를 굴리기 이후 구간으로 제한할 수 있었습니다.

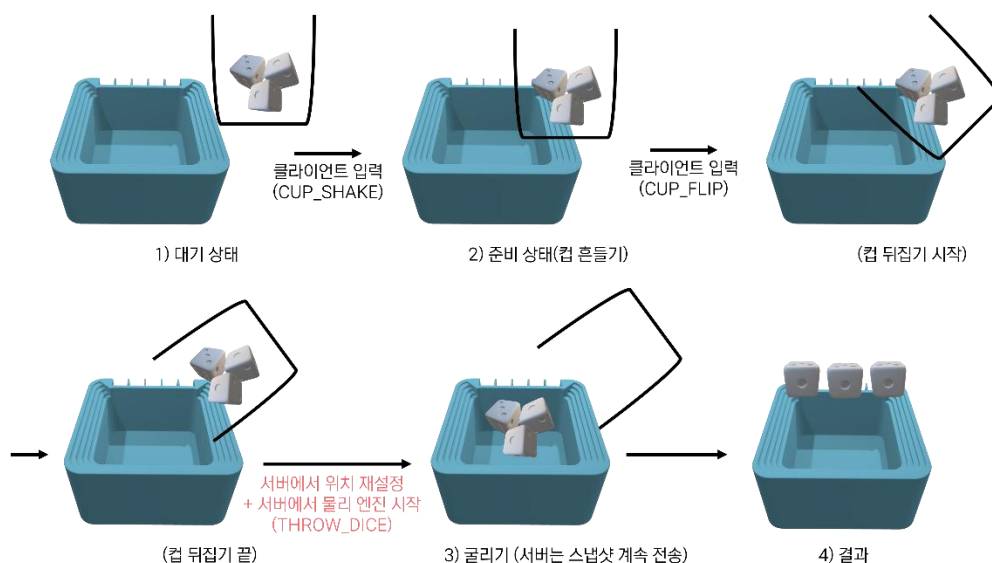


그림 29. 멀티플레이 환경 흐름

5.4. [트레이드오프] 서버로 옮기는 범위를 줄인 대신 그에 대한 비용이 있다.

1) 처음부터 서버 권위를 전제하고 개발했다면 치르지 않았을 비용이다.

이 문제가 발생한 이유 자체는 클라이언트에서 단독 실행을 전제로 개발을 진행한 뒤에 멀티플레이 환경을 구성했기 때문입니다. 처음부터 서버 환경에서 개발을 시작했다면 서버와 클라

이언트 경계를 나누고 메시지를 전달받아 전환 지점을 정하는 과정 필요없이 구현할 수 있었습니다. 기능을 구현하기 전에 실행 환경을 먼저 확정해야 한다는 점을 이 과정에서 확인했습니다.

2) 연출 단계마다 별도의 패킷을 정의해야 했다.

기존의 클라이언트 코드를 유지하기로 한 선택은 **패킷의 구조에서 비용이 드러났습니다**. 물리 연산과 연출을 모두 서버가 담당하는 구조였다면 패킷의 구조가 크게 클라이언트의 입력을 전달하는 패킷, 오브젝트의 Transform의 스냅샷을 전달하는 패킷 등으로만 구성되어 있을 것입니다. 그러나 컵을 흔들고 뒤집는 연출들이 이미 클라이언트에 단계별로 구현되어 있었기 때문에, 각 단계(SHAKE, FLIP 등)를 다른 클라이언트에도 동일하게 재생하려면 단계마다 별도의 패킷이 필요했습니다. S_CUP_SHAKE, S_CUP_FLIP과 같이 연출 단계에 대응하는 메시지가 추가로 정의된 것은 이러한 이유입니다.

```
void HandleJoin(BYTE* buffer, int32 len);
void HandleThrowDice(BYTE* buffer, int32 len);
void HandleSelectScore(BYTE* buffer, int32 len);
void HandleLeave(BYTE* buffer, int32 len);
void HandleInjectOrientation(BYTE* buffer, int32 len);
void HandleClearInject(BYTE* buffer, int32 len);
void HandleCupShake(BYTE* buffer, int32 len);
void HandleCupFlip(BYTE* buffer, int32 len);
void HandleDiceSelect(BYTE* buffer, int32 len);
void HandleDiceReturn(BYTE* buffer, int32 len);
void HandleDiceToCup(BYTE* buffer, int32 len);
```

그림 30. 다양한 패킷 종류로 인해 생기는 함수 목록

그림과 같이 메시지의 서버와 클라이언트가 다뤄야 할 메시지의 종류가 많았고, 그에 따라 패킷과 코드의 분량이 많아졌고 추후 개발에 유지 보수성을 떨어뜨릴 수 있는 문제가 있습니다.

3) 네트워크의 지연은 주사위의 움직임이 렉 걸린 것처럼 나타날 수 있다.

```

case State::NETWORK_DRIVEN:
{
    if (!m_transform) break;

    m_netLerpT += dt / NET_SNAPSHOT_INTERVAL;
    if (m_netLerpT > 1.f) m_netLerpT = 1.f;

    Vec3 pos = Vec3Lerp(m_netPrevPos, m_netTargetPos, m_netLerpT);
    Quat rot = QuatSlerp(m_netPrevRot, m_netTargetRot, m_netLerpT);

    m_transform->SetPosition(GameEngine::ToEngine(pos));
    m_transform->SetRotation(GameEngine::PhysicsQuatToEuler(rot));
    break;
}

```

그림 31. 클라이언트 update에서 위치/회전 동기화 코드

클라이언트는 서버가 전송한 스냅샷 사이를 보간하여 주사위를 표시하므로, 다음 스냅샷이 도착할 때까지 참조할 목표 상태가 없습니다. 만약 패킷 도착이 지연되면 주사위가 마지막으로 수신한 상태에 머무르게 되어, 화면에서는 주사위가 잠시 멈췄다가 다시 움직이는 것처럼 보일 수가 있습니다.

6. 결과 및 회고

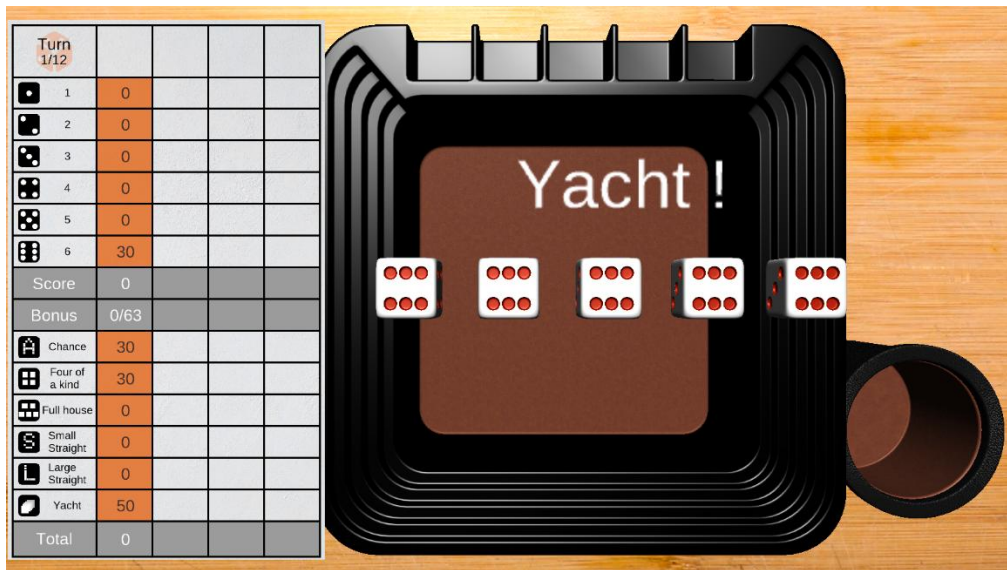


그림 32. 실제 게임 내 스크린샷

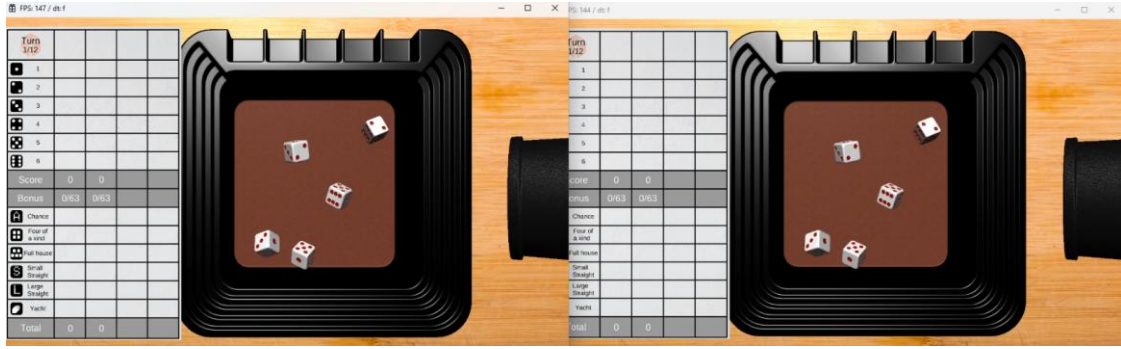


그림 33. 클라이언트 동기화 확인

My-YachtDice 프로젝트에서 힘과 회전 운동의 적용부터 충돌 감지, Contact 생성과 해결, 위치 및 회전 갱신까지 이어지는 물리 처리 구조를 구성했습니다. 이를 통해 여러 주사위가 낙하하고 서로 충돌하며 굴러가는 기본적인 강체 물리 시뮬레이션을 프로젝트에 적용했습니다.

다른 물체 두개 이상이 충돌할 때 접촉을 해결하는 Contact solver에서 기존에는 단 한번의 계산으로 처리하지 못한 속도가 프레임마다 누적되어 물체가 접촉면을 파고드는 문제가 있었습니다. 이 문제를 한 프레임에 여러 번 접촉 해결을 반복하는 방식으로 처리했습니다. 반복 과정에서 상대 속도가 0으로 수렴하며 반발이 사라지는 문제는 반복 이전의 상대 속도를 별도로 저장해 반발 계산에 적용하여 해결했습니다.

이렇게 구성한 물리 엔진을 기반으로 주사위의 Orientation을 직접 변경하지 않고 Torque를 이용해 원하는 자세로 유도하는 기능도 구현했습니다. P 제어에서 발생한 진동을 각 속도를 고려하는 PD 제어로 보완하여, 기존의 충돌과 회전 운동을 유지하면서 목표 Orientation으로 수렴하도록 구성했습니다.

결과적으로 프로젝트에서 필요했던 주사위의 물리적인 낙하와 충돌뿐만 아니라, 물리 시뮬레이션 안에서 결과를 원하는 방향으로 제어할 수 있는 구조까지 하나의 물리 파이프라인 안에서 구현했습니다.